

Запускать так: `./run.sh`.

Эвристики

Попытаемся найти перестановку вершин, на которой при жадном алгоритме выходит мало цветов. Далее, подобно эвристике с лекции будем переставлять группы цветов и перезапускать жадный алгоритм. После того как сойдёмся начнём локальный поиск по покраске. Теперь обо всём по порядку:

Рассмотрим перестановку вершин $1, 2, \dots, n$. Для неё запустим жадный алгоритм: будем красить вершины в порядке перестановки так, чтобы не возникало противоречий с уже покрашенными. После того как граф будет покрашен, рассмотрим размер каждой получившейся группы вершин одного цвета и переставим вершины в таком порядке, что сначала будут идти вершины самого частого цвета, потом цвета чуть реже, и так далее, в конце будет самый редкий цвет. На получившейся перестановке снова запустим тот же жадный алгоритм. Так будем повторять до сходимости.

В дословно такой реализации получились следующие значения: 9, 23, 23, 100, 20, 122. Что дало 3 балла следующим образом: 0, 0, 0, 0, 0, 3.

Таким образом мы хотели получить раскраску, у которой популярность цветов будет в отсортированном порядке, тем самым избежать часть симметрий. Проблема такого метода заключается в том, что работать до сходимости на самом деле оказывается не очень долго, а именно было потрачено 1, 1, 2, 1, 3, 5 итераций на тест соответственно.

Также было попробовано запускаться из множества случайных начальных перестановок, но даже 10000 случайно выбранных перестановок не дали улучшения, все значения числа цветов на тестах остались прежними.

Также была попробована и другая сортировка, а именно сортировать по убыванию степеней вершин, она дала значительно лучший результат, получились метрики: 7, 20, 21, 97, 18, 121. То есть 15 баллов следующим образом: 3, 3, 3, 0, 3, 3.

После этого был написан ещё один конструктив.

Введём следующее отношение порядка на ответах на покраску: выпишем массив того, сколько вершин какого цвета у нас есть и отсортируем его от большего к меньшему. Будем считать раскраски в которых в последнем классе меньше вершин лучше чем другие раскраски в такое же количество цветов. Мотивация этого: требуется найти другой цвет для меньшего числа вершин. Уже имеющаяся эвристика эвристика проталкивает вершины в более ранние цветовые классы в порядке сортировки, так что будем считать, что она минимизирует самый последний.

Научимся перекрашивать вершину из последнего класса: возьмём самую последнюю вершину v в расстановке и посмотрим на всех её соседей. Нам нужно найти новый цвет для v , а если v цвета k , то у неё есть соседи всех меньших цветов. Тогда для каждого из цветов $1, 2, \dots, k - 1$ возьмём все вершины из окружения v этого цвета и проверим, может быть их можно перекрасить в какой-то другой цвет. Для этого достаточно перекрасить каждую по отдельности, так как сами вершины одного цвета в корректной покраске, а значит образуют независимое множество, то есть их новые цвета не могут давать противоречие в раскраске между друг-другом, только с остальными вершинами. Если так вышло, что всех соседей v одного какого-то цвета, скажем i , можно перекрасить, то перекрасим v в цвет i , а всех соседей v цвета i в тот, который был найден как возможный, получим раскраску лучше, снова передадим её нашей эвристике, покрасим набор и повторим процедуру. Если процедура

остановилась, то попробуем и для других вершин последнего цвета. Если и для них ничего нельзя изменить, то остановимся.

Этот подход улучшил метрики, вышли следующие результаты: 7, 20, 19, 94, 17, 113. То есть 18 баллов, следующим образом: 3, 3, 3, 3, 3, 3. Заметим, что на всех тестах этот процесс сошёлся, то есть проделывать такие улучшения больше было нельзя.

Beam Search

Далее был добавлен *Beam Search*. Запустив все наши эвристики найдём какую-нибудь стартовую точку, из неё начнём идти с помощью *Beam Search*. Выберем метрику, как и говорилось ранее, следующим образом: посмотрим на массив количеств всех цветов. Сперва сравним его по размеру, если они не равны, то меньше цветов — лучше. Если количества цветов равны, то лучше тот, у кого меньше вершин в наименее популярном цвете, при равенстве — в цвете на 1 популярнее и так далее. Изменения будем делать следующим образом: возьмём вершину последнего цвета и попытаемся поменять её с кем-то из её соседей (изменение в распределении цветов будет только если в перестановки свапнуть смежные вершины, а в первую очередь хочется избавиться от последнего цвета). Если изменение улучшает разбиение то сразу же примем его, иначе закинем все возможные разбиения после 1 такого шага в кучу, где если мы поменяли (u, v) , то запомним, что надо сделать *swar* вершины u с кем-то, или вершины v с кем-то, в конце выделим из кучи несколько самых лучших найденных разбиений, и продолжим поиск для них, если после пути некоторой длины (выбрана 5) не удалось ничего улучшить, то будем считать что мы в локальном минимуме и остановимся.

Этот подход улучшил метрики на средних тестах (на пятом и шестом при выбранных константах он не сумел сделать один шаг, уложившись во время). Метрики составили:

7, 18, 18, 85, 17, 113.

То есть всё те же 18 баллов 3, 3, 3, 3, 3, 3.

Сейчас наш алгоритм быстро сходится на первых пяти тестах, так как окрестность Beam Search задана явно. Изменим это: пусть если мы не справились найти ничего лучше, то мы пойдём искать дальше, пока не иссякнет время. В таких предположениях алгоритм всё равно делал в пределах 20 итераций. Это свидетельствовало о том, что окрестность была слишком маленькой.

Настроим параметры следующим образом: дадим 500 глубины, что в целом должно разрешать любые изменения. У нас есть два параметра: время раскрытия одного слоя, что важно для больших тестов и количество потомков переходящих на последующие слои. Слишком большое время застопорит алгоритм, мы будем бесконечно раскрывать один слой. Слишком большая константа даст слишком много элементов для раскрытия и часть данных потеряется. При этом слишком маленькое время и слишком маленькая константа уменьшат окрестность и можно будет потерять решения. Будем смотреть на метрики, а иначе на количество итераций, так как больше итераций означает, что нам хватило окрестности.

После окрестности в 150 метрики улучшаться перестали. При этом время играло какую-либо роль только для шестого теста, который ни при каком времени не делает больше одной итерации.

На данный момент метрики составляют

7, 18, 16, 83, 16, 113.

Что даёт 22 балла, как 3, 3, 5, 3, 5, 3.

Кол-во, время	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
1с, 10	7 цв., 2 ит.	18 цв., 5 ит.	18 цв., 8 ит.	85 цв., 36 ит.	17 цв., 1 ит.	113 цв., 1 ит.
1с, 30	7; 5	18; 5	17; 15	85; 36	17; 1	113; 1
1с, 60	7; 5	18; 10	17; 16	84; 45	16; 13	113; 1
1с, 100	7; 5	18; 11	16; 28	84; 45	16; 24	113; 1
1с, 150	7; 5	18; 11	16; 28	83; 50	16; 41	113; 1
1с, 200	7; 5	18; 11	16; 28	83; 54	16; 41	113; 1
2с, 160	7; 5	18; 11	16; 28	83; 50	16; 41	113; 1
2с, 160	7; 5	18; 11	16; 28	83; 50	16; 41	113; 1

Локальный поиск

Даже BeamSearch не сумел найти точку лучше на глубине 500 для нашего локального поиска, значит у него слишком маленькая окрестность такими преобразованиями, добавим возможных локальных изменений, чтобы увеличить окрестность.

Снова начнём отталкиваться от перестановки в которой вершины одного цветового класса идут подряд. По совету Михаила Симакова попробуем не только перестановку в порядке убывания размеров классов, но и в обратном и случайной порядке.

Было реализованно следующее: сперва попробуем переставить вершины, ставя более большие классы первыми, если это не вышло, то попытаемся поставить более редкие классы первыми (сортировки по убыванию числа вершин и возрастанию соответственно). Если ни то, ни другое не сработало, то переставим классы в случайном порядке. Если получилось улучшить, то примем изменение. Тут появился параметр сколько раз стоит пробовать переставлять вершины, его подберём, так как не хочется делать лишние действия, при этом надо получить нужную перестановку.

Кол-во перестановок	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
10	7	19	18	84	16	113
100	7	18	17	84	15	112
500	6	18	17	82	15	106
2000	6	18	16	82	14	103
5000	6	18	16	79	14	103
10000	6	18	16	79	14	103

После 5000 итераций улучшения закончились, на них и остановимся.

На данный момент метрики

6, 18, 16, 79, 14, 103

Что даёт 24 балла: 5, 3, 5, 3, 5, 3.

В целом перебирать разные перестановки дало пользу, однако все их мы перебрать не может, попробуем написать аналог сортировки пузырьком на перестановках, будет пробовать переставить два соседних класса, если это улучшит метрику. Вероятность сейчас от части корректностная, так как всё ещё делается несколько случайных итераций, поставив на 20 перезапусков все тесты кроме шестого, на котором время сильно больше в течении 30 секунд на каждом тесте были получены следующие метрики:

6, 17, 16, 79, 14, 102.

Что даёт 26 баллов как 5, 5, 5, 3, 5, 3.

Ещё одно локальное улучшение, которые мы добавим, чтобы увеличить окрестность будет следующим: попытаемся на подобии первой части отдать максимум вершин другим цветам, то есть возьмём какой-то цвет и попытаемся поставить его последним.

Вставив эту эвристику до случайной перестановки метрики составили

7, 18, 16, 79, 14, 103.

То есть ухудшились, видимо это перестановка сводит быстрее в локальный минимум, который не может улучшить случайная перестановка. Попробуем исправить это двумя способами:

Во первых попробуем запускать последнюю эвристику с перестановкой класса в конец в последнюю очередь, чтобы не ухудшать результаты, которые мы имели раньше. Это дало метрики

6, 17, 16, 79, 14, 102.

Что действительно вернуло старые результаты.

Второй вариант — сменить функцию, по которой мы сравниваем, чтобы не требовать более узкие конструкции, с очень маленьким последним классом. Попробуем по набору распределения цветов $(a_1, a_2, \dots, a_n)$ вычислять $\sum_{i=1}^n a_i i^\alpha$, где α — подберём.

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.5	6	18	16	79	16	107
1	6	18	16	81	15	110
2	6	17	16	81	15	110
3	6	18	16	82	15	108
5	7	18	16	81	15	107

Попробуем взять лучшую с точки зрения баллов $\alpha = 2$ и всё же вынести блок с проталкиванием в конец отдельно от остального локального поиска. В этом случае метрики составили

6, 17, 16, 80, 15, 106.

То есть не хуже, при этом эта функция существенно замедляет код, так что отключим её.

Подберём и лучшее α в случае без неудачной функции

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.33	7	17	17	80	16	107
0.5	6	17	16	80	16	109
1	6	17	17	80	15	108
2	6	17	16	81	15	110
3	6	18	17	81	15	108
5	6	17	16	80	15	110

По метрикам лучше всего оказались $\alpha = 0.5, 2$, если ухудшение при альфа меньших 0.5 и больше 2, а также локально в единице. Возьмём $\alpha = 2$.

Отжиг

Поскольку метрика стала численной, а не векторной, то на ней можно ввести конкретную дельту и написать отжиг.